

NUS-ISS

*Pattern Recognition using
Machine Learning System*



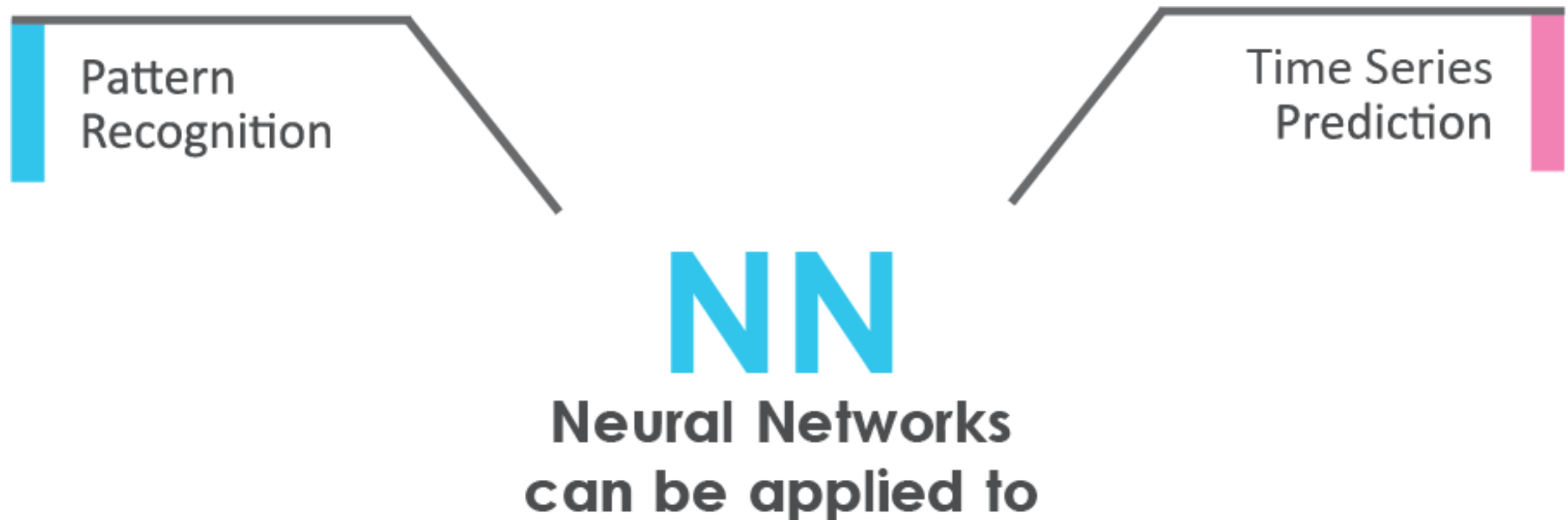
Day 3 (II) - Solving temporal sequential problems using recurrent neural networks

by Dr. Gary Leung

When time is a factor

The other application

with time

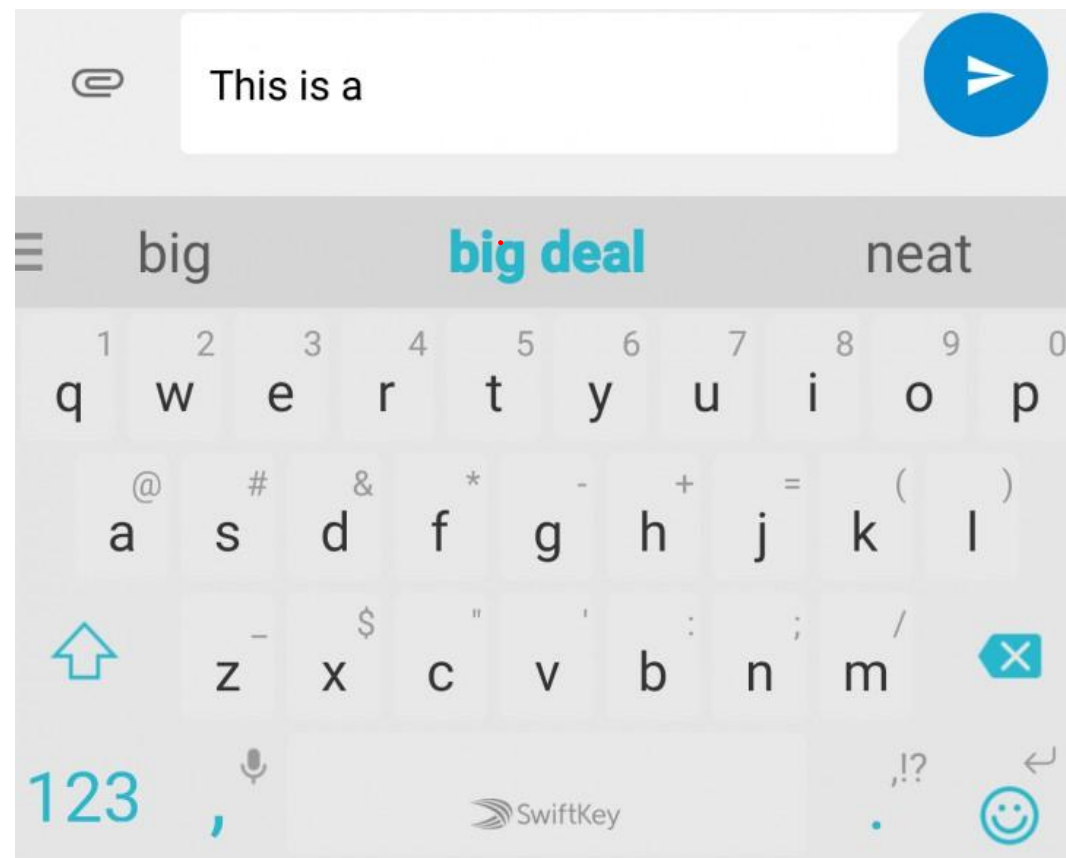


Source: <http://www.cvisiontech.com/resources/ocr-primer/ocr-neural-networks-and-other-machine-learning-techniques.html>

The related inputs

Not independent

- So far the networks introduced assume inputs are independent from each other
- Implication: the inputs that came before and the inputs that will come after have no relationship
- But for some tasks/problems, this is not true
- E.g. if. you want to predict which word comes next in a sentence, you better know what have been typed/ said before

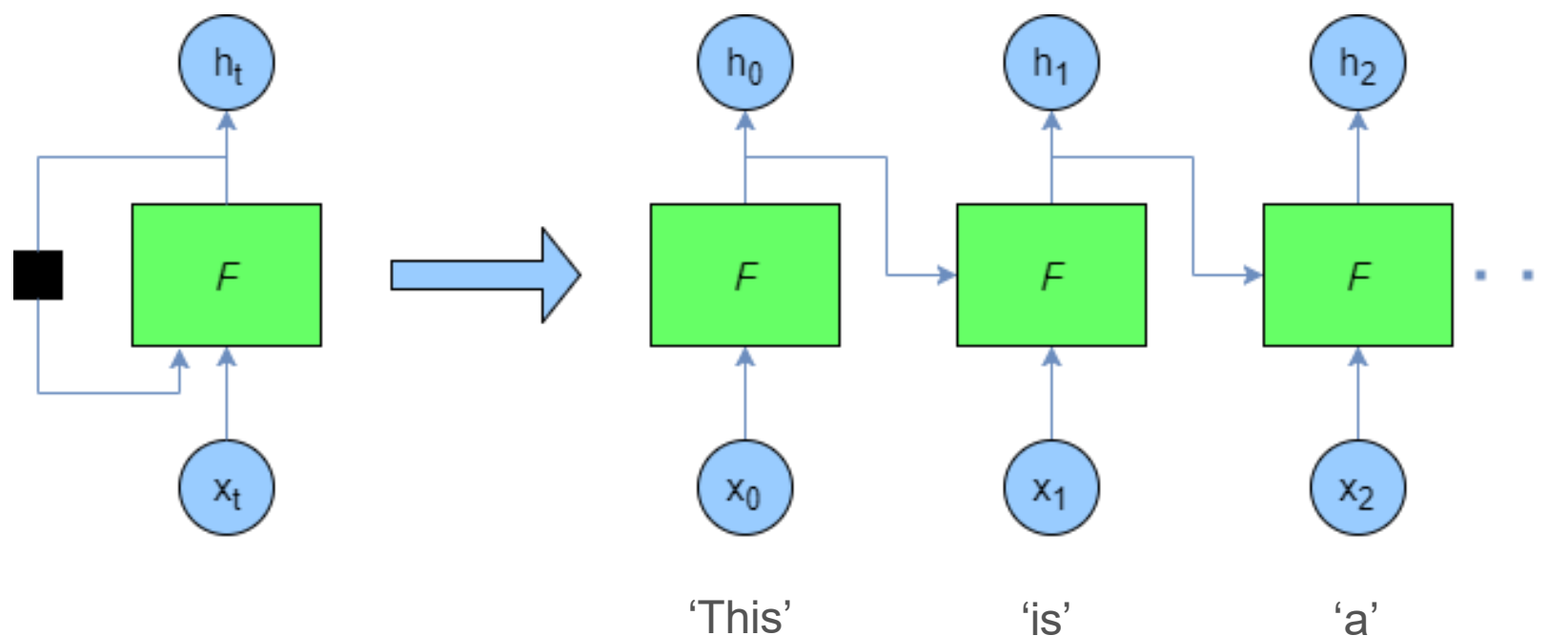


Source: <https://www.androidpolice.com/2015/11/06/swiftkey-update-to-v6-0-with-redesigned-settings-menu-double-word-prediction-and-more/>

Recurrent neural network

In short, RNN

- Recurrent neural network: a network that performs the same calculation on elements/segments from a sequence
- The output of a current element/segment depends on the hidden state carried over from previous steps



Source: <https://adventuresinmachinelearning.com/recurrent-neural-networks-lstm-tutorial-tensorflow/>

CNN vs RNN

Comparison

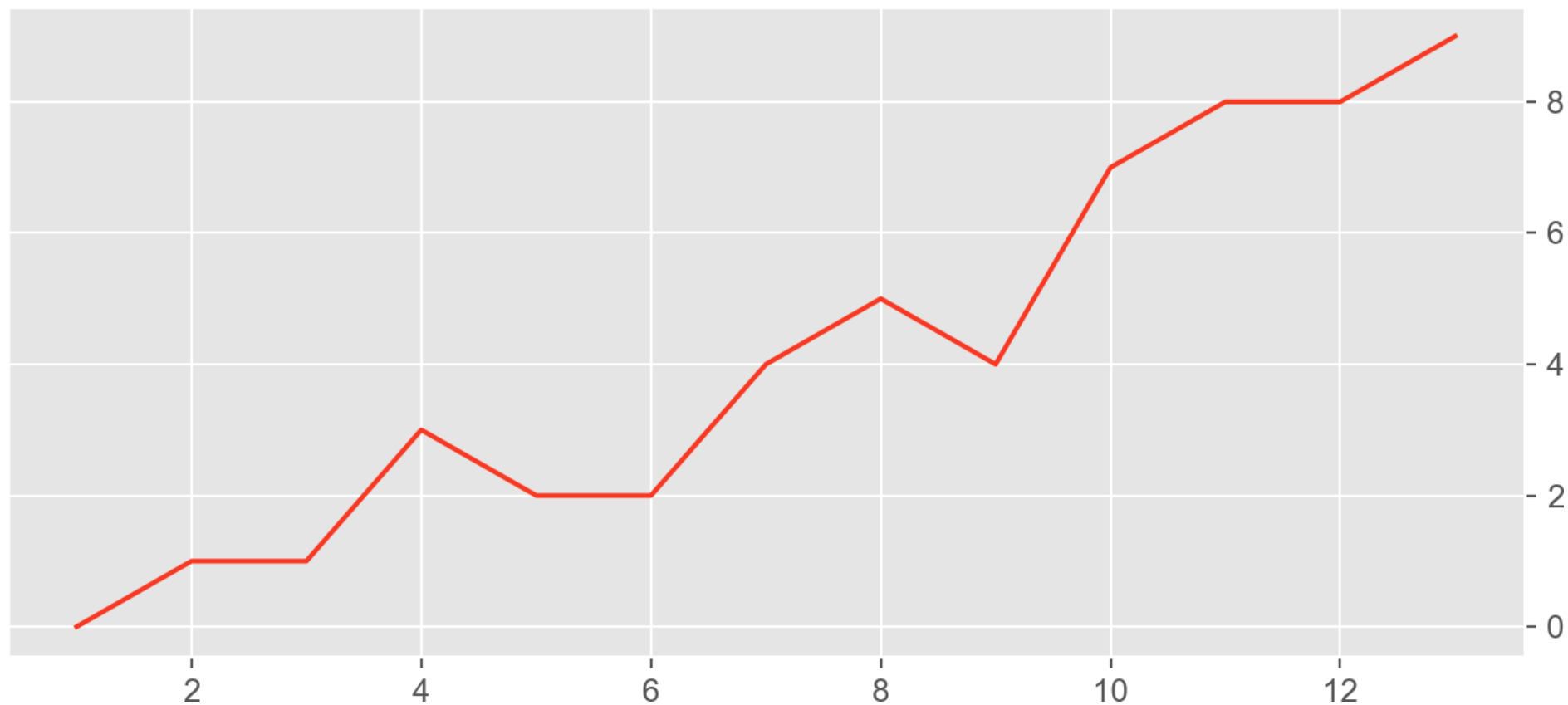
	CNN	RNN / LSTM
Usage	Suitable for spatial data, e.g. image, video	Suitable for temporal data (sequential data), e.g. text, speech
Capability	Considered more powerful than RNN	Less powerful and slower in calculation
Input	Take fixed size inputs and generate fixed size outputs	Can handle arbitrary input/output lengths
Nature	Use local connectivity pattern (through 2D convolution)	Use time series information

Time step

Prediction

- Assume we have a series of 13 values, and we would like to predict the next value

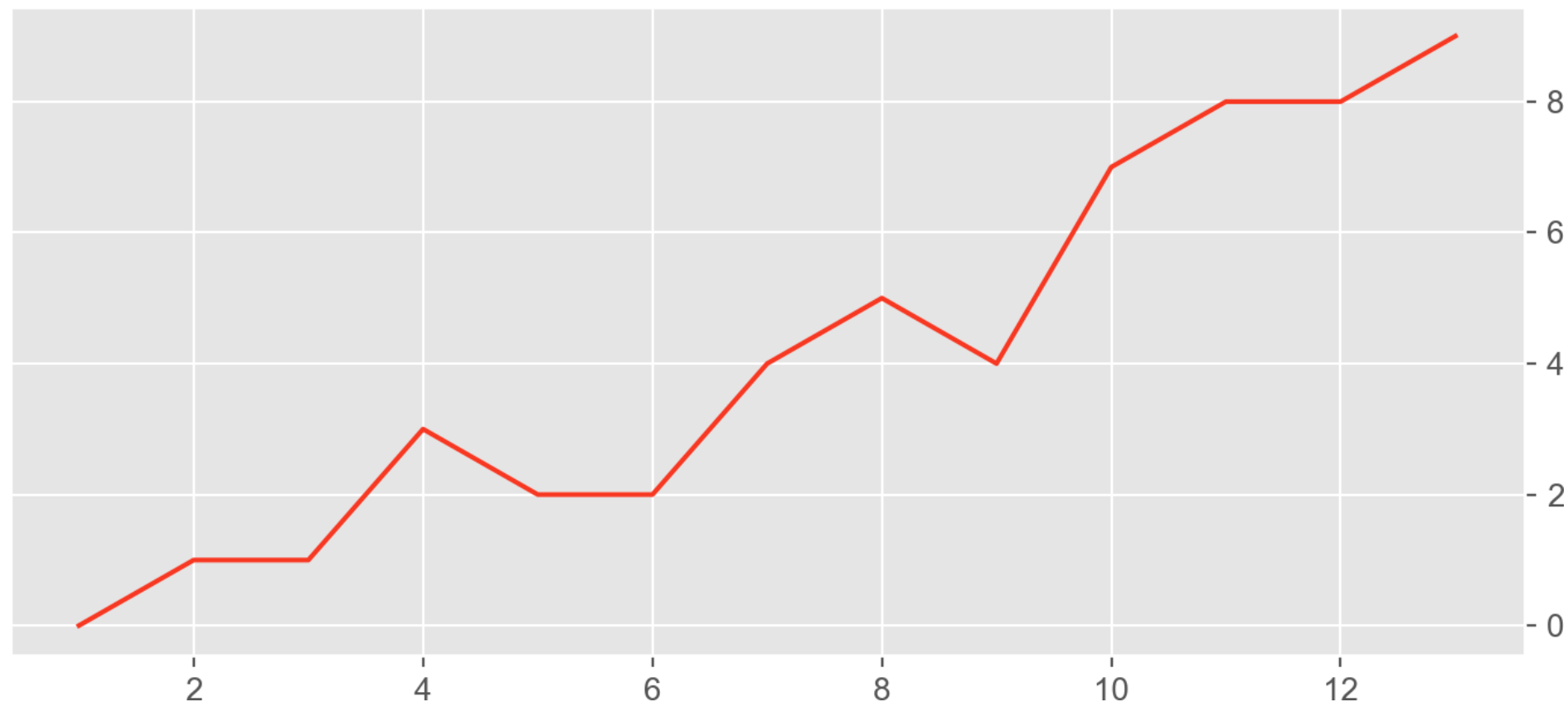
0, 1, 1, 3, 2, 2, 4, 5, 4, 7, 8, 8, 9, ?



Time step

Make segments

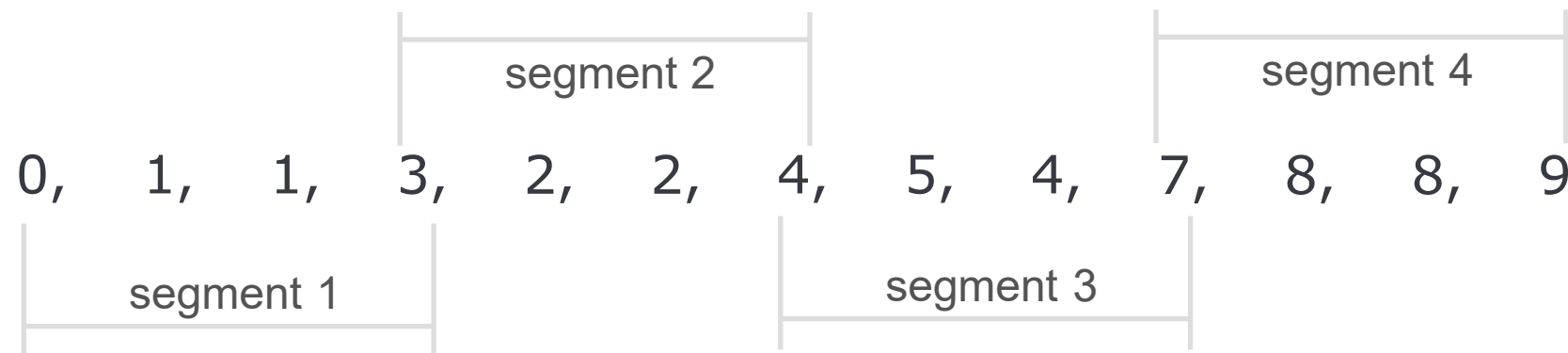
- For time series problems, we don't feed in the entire series into the neural network.
- Instead, we chop the series into segments, each segment with a fixed amount of time steps, called length



makeSteps

The procedure

- Assume we want to have a length of 4, and a distance of 3



- The preprocessed input:

0,	1,	1,	3
3,	2,	2,	4
4,	5,	4,	7
7,	8,	8,	9

makeSteps

The procedure

- Assume we want to have a length of 5, and a distance of 3



- The preprocessed input:

0, 1, 1, 3, 2
3, 2, 2, 4, 5
4, 5, 4, 7, 8

Exercise

The procedure

- Write a function that can generate the desired preprocessed input from a 1D series/signals with the below signature

```
> def makeSteps(dat, length, dist):
```

0, 1, 1, 3, 2, 2, 4, 5, 4, 7, 8, 8, 9

- length 4, distance 3

0, 1, 1, 3
3, 2, 2, 4
4, 5, 4, 7
7, 8, 8, 9

- length 5, distance 3

0, 1, 1, 3, 2
3, 2, 2, 4, 5
4, 5, 4, 7, 8

Dot product

The working

- What is the output of

$$\mathbf{W} \cdot \mathbf{x}$$

- Assume the $\mathbf{W}$ is a $m \times n$ matrix, $\mathbf{x}$ is $n \times 1$ matrix, the matrix-vector dot product is

$$\mathbf{W} \cdot \mathbf{x} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} w_{11}x_1 + w_{12}x_2 + \cdots + w_{1n}x_n \\ w_{21}x_1 + w_{22}x_2 + \cdots + w_{2n}x_n \\ \vdots \\ w_{m1}x_1 + w_{m2}x_2 + \cdots + w_{mn}x_n \end{bmatrix}$$

- m determines the size of the product output, i.e. the output feature size

Dot product

The working

- What is the output of

$$\mathbf{W} \cdot \mathbf{x}_1$$

- Assume

$$\mathbf{W} = \begin{bmatrix} 1 & 0 & 0 & 1 & -1 \\ 2 & 0 & 0 & -1 & 1 \end{bmatrix}$$

- and we know

$$\mathbf{x}_1 = [0 \quad 1 \quad 1 \quad 3 \quad 2]$$

Dot product

The working

- What is the output of

$$\mathbf{W} \cdot \mathbf{x}_1$$

- Assume

$$\mathbf{W} = \begin{bmatrix} 1 & 0 & 0 & 1 & -1 \\ 2 & 0 & 0 & -1 & 1 \end{bmatrix}$$

- and we know

$$\mathbf{x}_1 = [0 \ 1 \ 1 \ 3 \ 2]$$

$$\mathbf{W} \cdot \mathbf{x}_1 = \begin{bmatrix} 1 & 0 & 0 & 1 & -1 \\ 2 & 0 & 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 1 \\ 3 \\ 2 \end{bmatrix} = \begin{bmatrix} 1 \times 0 + 0 \times 1 + 0 \times 1 + 1 \times 3 + (-1) \times 2 \\ 2 \times 0 + 0 \times 1 + 0 \times 1 + (-1) \times 3 + 1 \times 2 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

Multiplication

Element-wise

- The output of element-wise multiplication is

$$\mathbf{v} \odot \mathbf{x} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix} \odot \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} v_1 x_1 \\ v_2 x_2 \\ \vdots \\ v_n x_n \end{bmatrix}$$

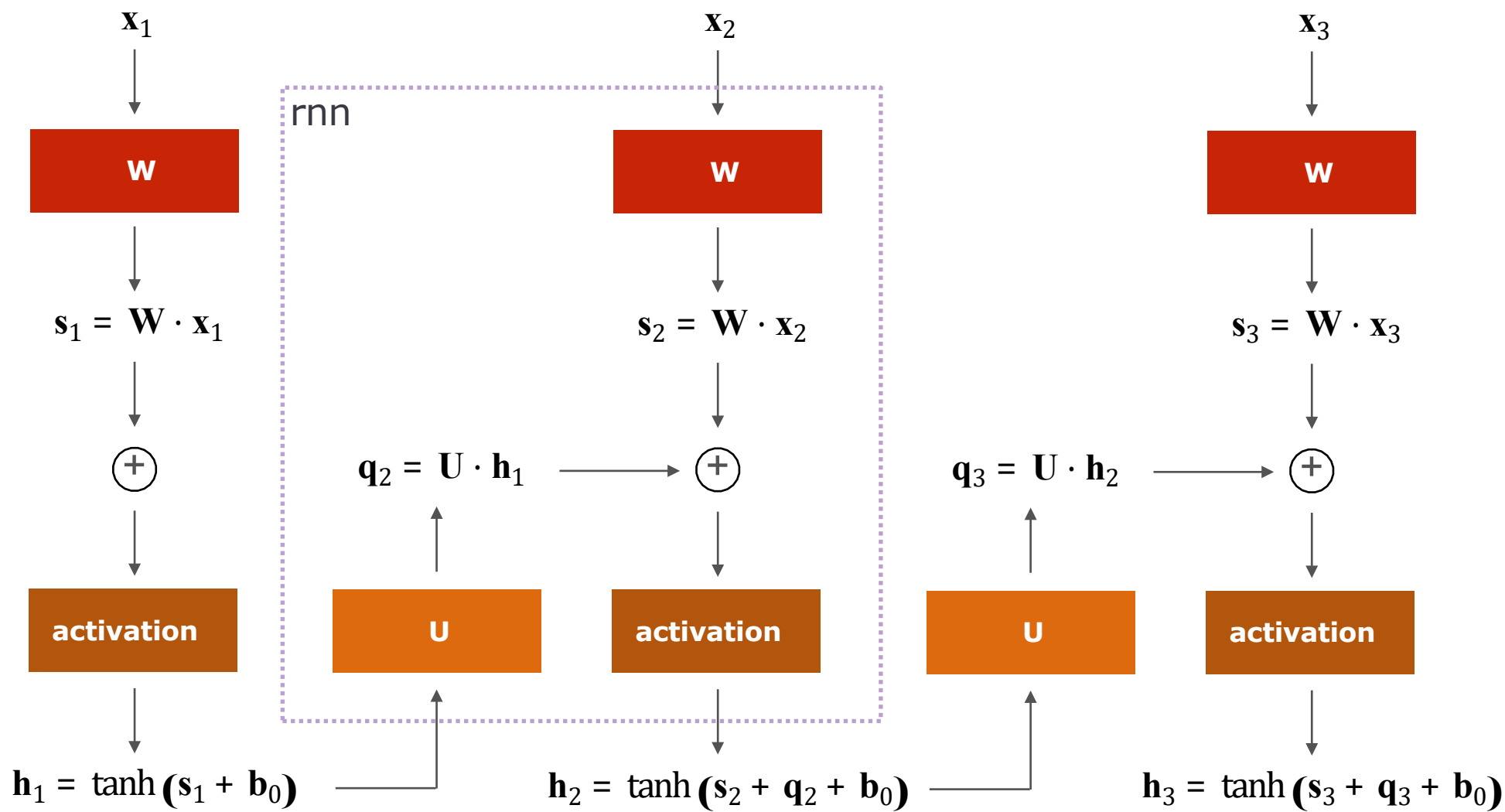
- The length of $\mathbf{v}$ and $\mathbf{x}$ must be equal

RNN

The working

- For RNN, the we feed the input segment by segment or row by row

0,	1,	1,	3,	2	→	$\mathbf{x}_1$
3,	2,	2,	4,	5	→	$\mathbf{x}_2$
4,	5,	4,	7,	8	→	$\mathbf{x}_3$

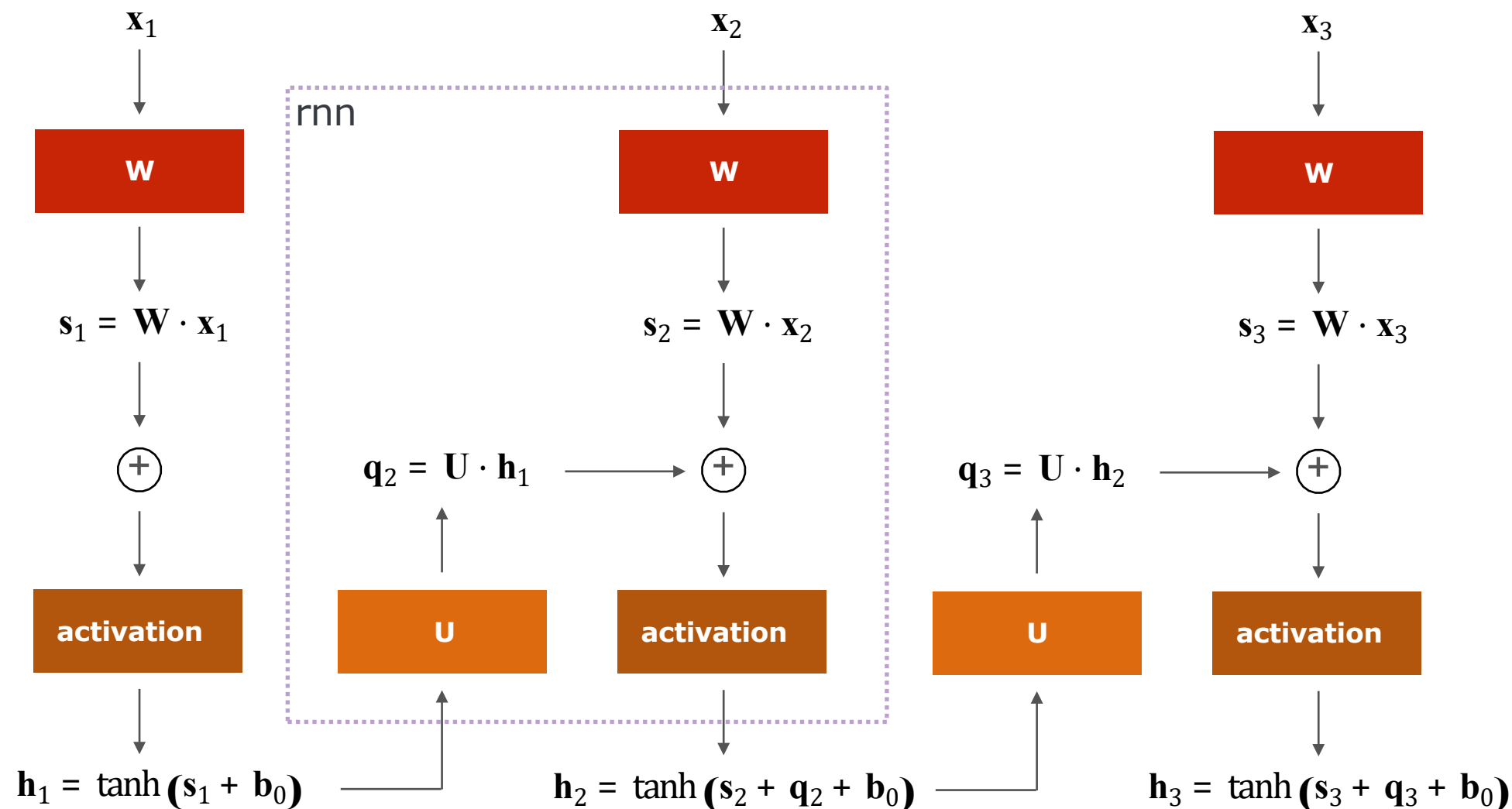


RNN

The working

- **Note:**

- The same $\mathbf{U}$ and $\mathbf{W}$ for $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$
- $\mathbf{U}$ and $\mathbf{W}$ are changed **only** during updating phase of training
- $\mathbf{U}$ is a square matrix
- For simplicity, output layer $y_t = \mathbf{V} \cdot \mathbf{h}_t$ is omitted



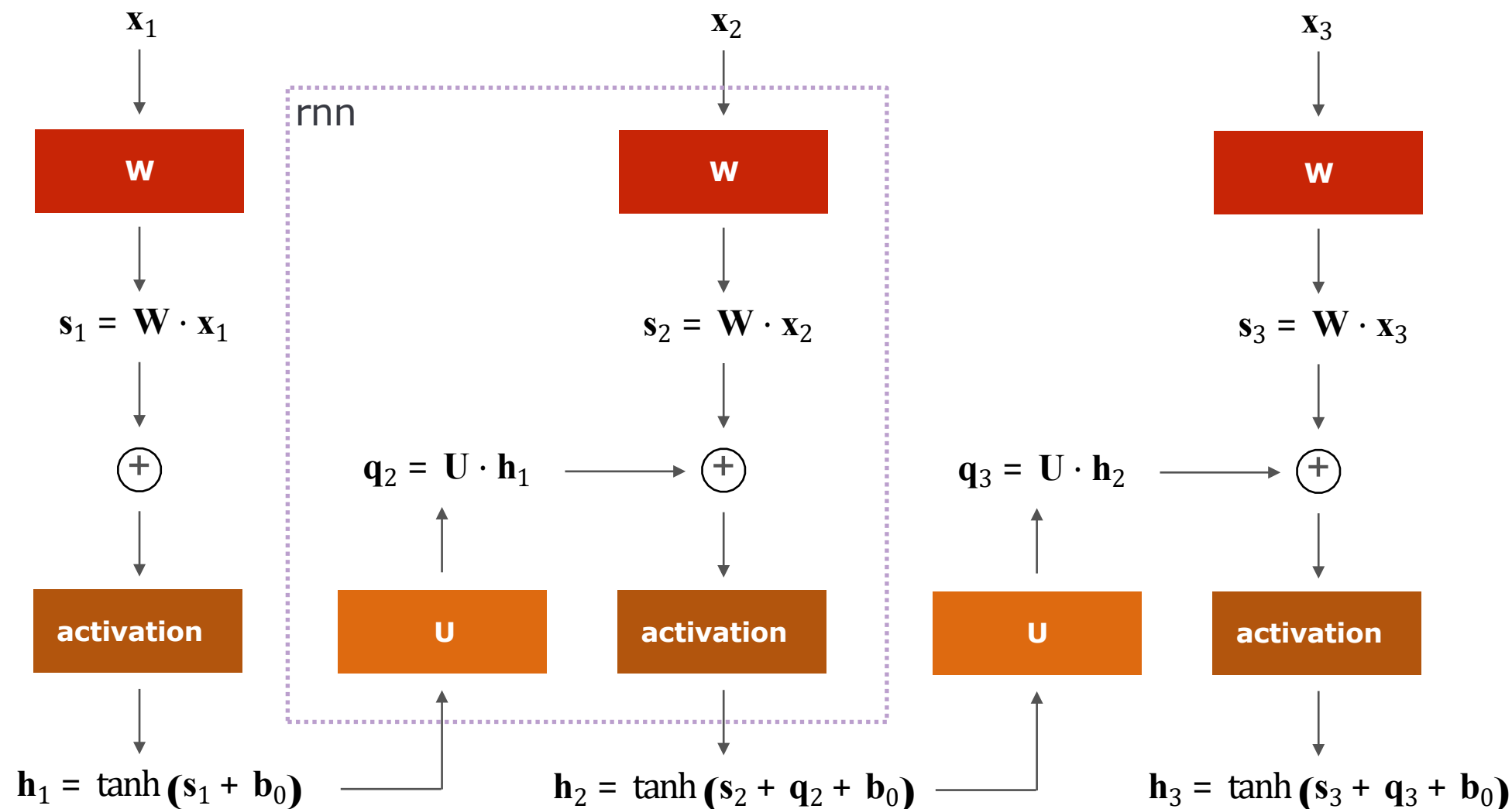
RNN

The working

- **Reflection: What are the matrix dimensions in sentiment classification?**

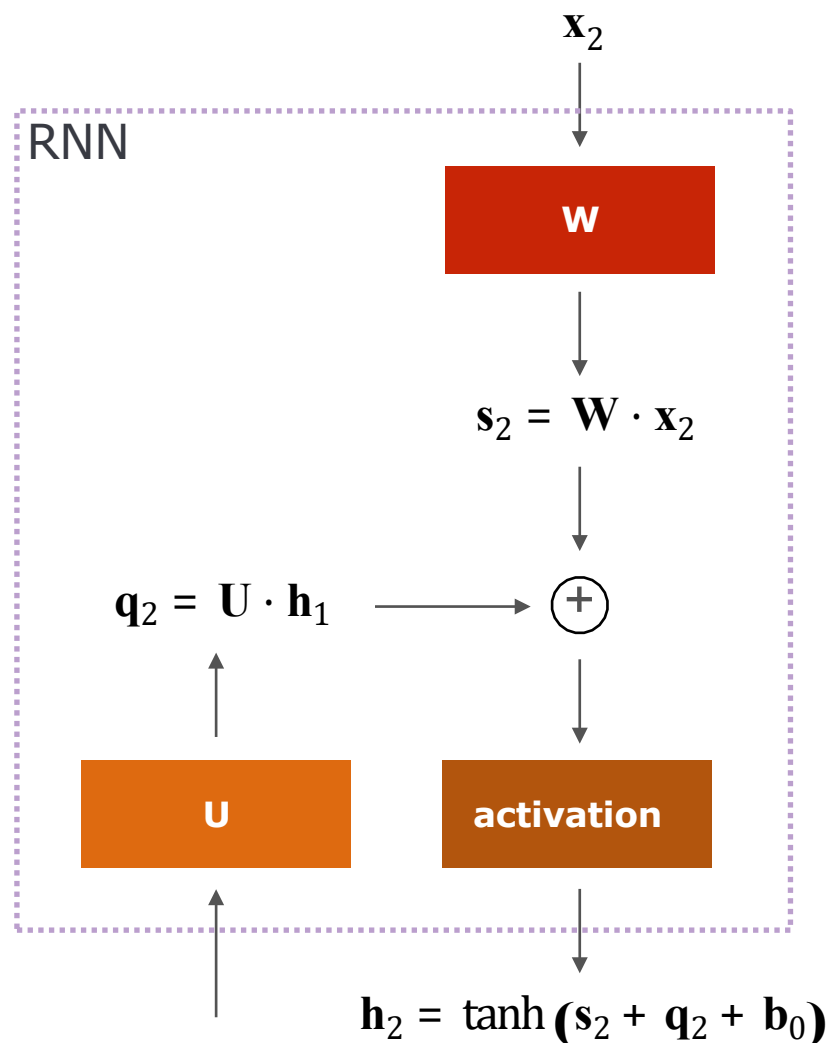
Suppose we train an RNN for **sentiment classification** with:

- **Input features** = 3 (e.g., sleep hours, work hours, exercise hours)
- **Hidden size** = H (chosen hyperparameter, e.g., 8, 16, ...)
- **Output classes** = 4 (e.g., *Happy*, *Sad*, *Stressed*, *Relaxed*)



RNN

The problem



- Ideally, we want RNN to have long memories, so that it can connect relationships among data points far before and after a point of interest
- If this is possible, good for language understanding / translation, or how events in stock market correlate to each other
- But, in RNN, the more we perform recurrent operation, mathematically, that is equivalent to adding more layers to the network
- So vanishing gradient[§] comes in ...

[§] Please refer to <https://medium.datadriveninvestor.com/how-do-lstm-networks-solve-the-problem-of-vanishing-gradients-a6784971a577>

RNN

How to build RNN in Keras

- Use [SimpleRNN](#) for RNN layers

```
> from tensorflow.keras.layers import Input
> from tensorflow.keras.layers import SimpleRNN
> from tensorflow.keras.layers import Dense
> from tensorflow.keras.models import Model

> inputs      = Input(shape=(16,64))
> y           = SimpleRNN(32)(inputs)
> y           = Dense(1, activation='sigmoid')(y)

> model       = Model(inputs=inputs,outputs=y)

> model.summary()
```

- A sample refers to one sequence from the dataset. How many segments (time steps) are there in a single sample?
- What is the length of each segment (features per step)?
- What is the size of the output vector from the RNN layer?

RNN

How to build RNN in Keras

The output layer determines the task. Use $\longrightarrow$
`Dense(1, activation='linear') + MSE`
for **regression**, `Dense(1, activation='sigmoid') +`
binary cross-entropy for **binary classification**,
or `Dense(k, activation='softmax') +` categorical
cross-entropy for **multi-class**.

```
> from tensorflow.keras.layers import Input
> from tensorflow.keras.layers import SimpleRNN
> from tensorflow.keras.layers import Dense
> from tensorflow.keras.models import Model

> inputs = Input(shape=(16,64))
> y = SimpleRNN(32)(inputs)
> y = Dense(1, activation='sigmoid')(y)

> model = Model(inputs=inputs,outputs=y)

> model.summary()
```

Layer (type)	Output Shape	Param #
=====		
input_1 (InputLayer)	(None, 16, 64)	0

simple_rnn (SimpleRNN)	(None, 32)	3104

dense (Dense)	(None, 1)	33
=====		
Total params: 3,137		
Trainable params: 3,137		
Non-trainable params: 0		

LSTM

from Francois Chollet's perspective

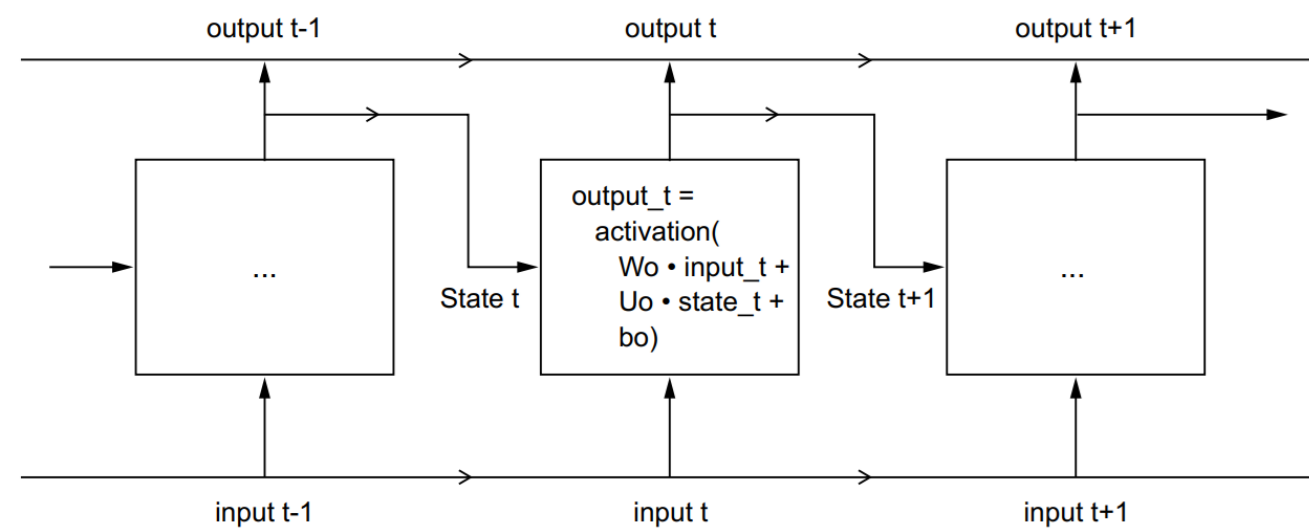


Figure 10.8 The starting point of an LSTM layer: a SimpleRNN

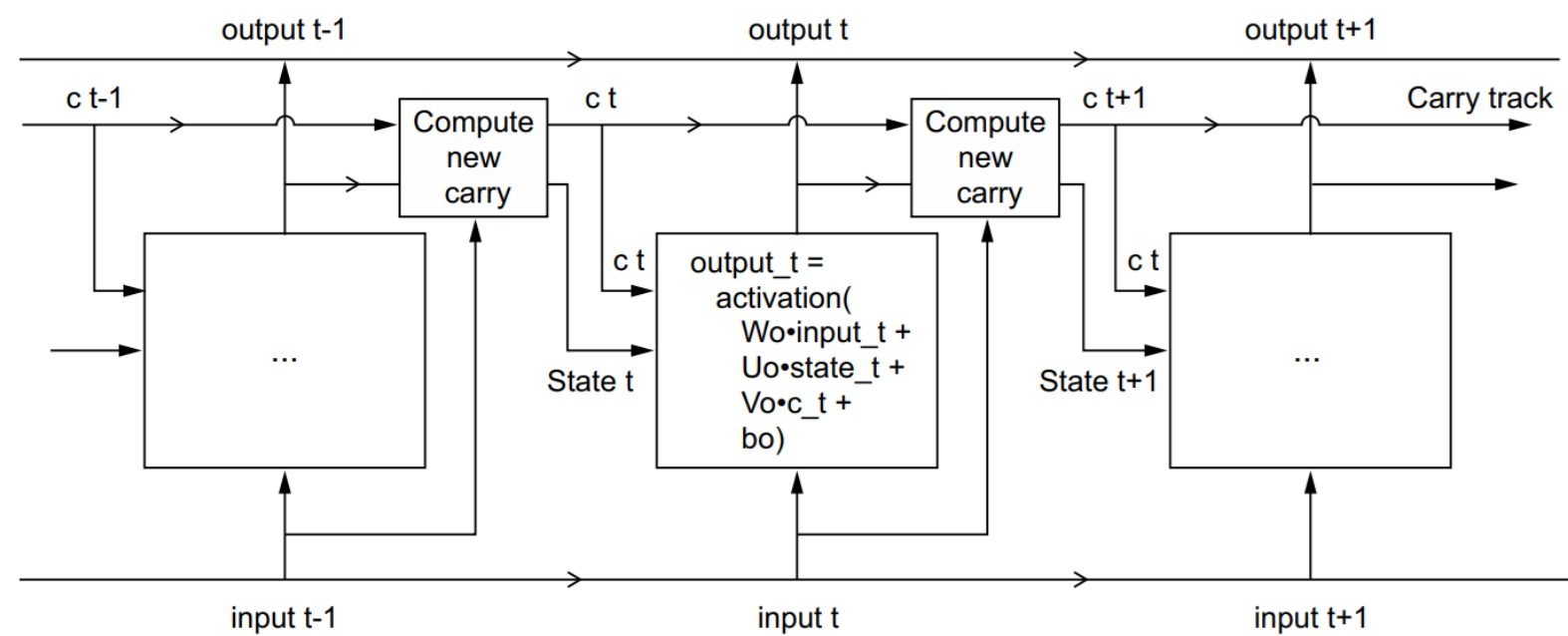


Figure 10.10 Anatomy of an LSTM

Source: “Deep Learning with Python” by Francois Chollet

LSTM

The internal working of 'carry'

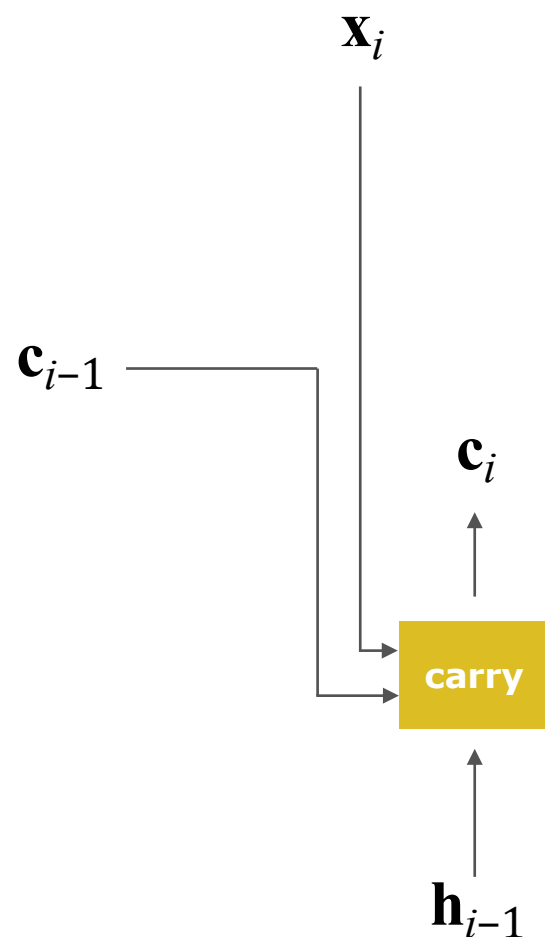
- First we define

$$\mathbf{w}_i = \sigma(\mathbf{U}_w \cdot \mathbf{h}_{i-1} + \mathbf{W}_w \cdot \mathbf{x}_i + \mathbf{b}_w)$$

$$\mathbf{f}_i = \sigma(\mathbf{U}_f \cdot \mathbf{h}_{i-1} + \mathbf{W}_f \cdot \mathbf{x}_i + \mathbf{b}_f)$$

$$\mathbf{h}_i = \sigma(\mathbf{U}_h \cdot \mathbf{h}_{i-1} + \mathbf{W}_h \cdot \mathbf{x}_i + \mathbf{b}_h)$$

- σ is the sigmoid function, and let's denote $\odot$ as element-wise multiplication, and we have



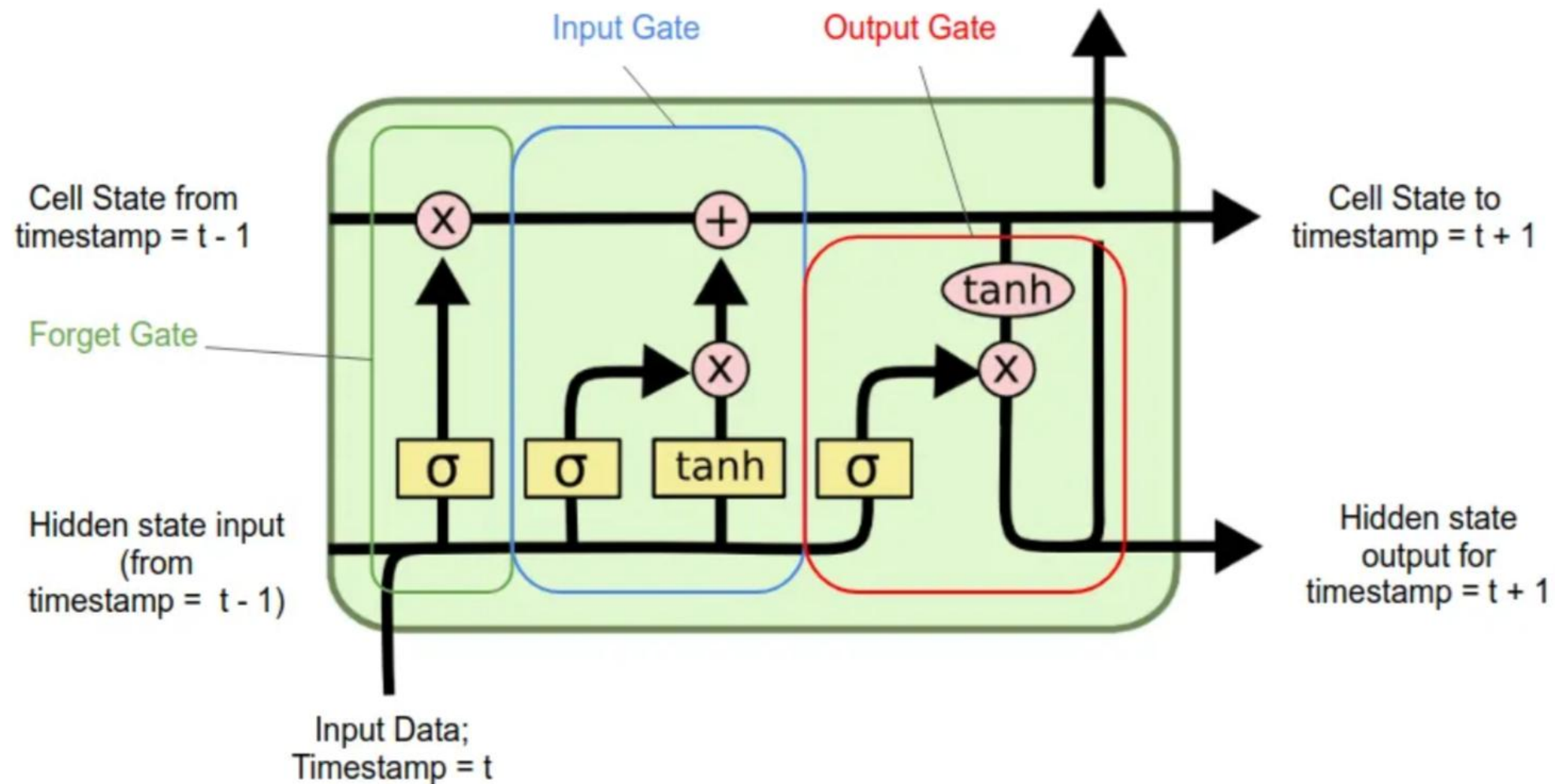
$$c_i = f_i \odot c_{i-1} + i_i \odot \tilde{c}_i$$

Forget gate: How much of my previous memory should I keep?

Input gate: How much of my previous memory should I keep?

- Further reading: <https://medium.com/analytics-vidhya/lstms-explained-a-complete-technically-accurate-conceptual-guide-with-keras-2a650327e8f2>

A single LSTM cell



Source: <https://medium.com/analytics-vidhya/lstms-explained-a-complete-technically-accurate-conceptual-guide-with-keras-2a650327e8f2>

LSTM

How to build LSTM in Keras

The output layer determines the task. Use `Dense(1, activation='linear') + MSE` for **regression**, `Dense(1, activation='sigmoid') + binary cross-entropy` for **binary classification**, or `Dense(k, activation='softmax') + categorical cross-entropy` for **multi-class**.

```
> from tensorflow.keras.layers import Input
> from tensorflow.keras.layers import LSTM
> from tensorflow.keras.layers import Dense
> from tensorflow.keras.models import Model

> inputs = Input(shape=(16,64))
> y = LSTM(32)(inputs)
> y = Dense(1, activation='sigmoid')(y)

> model = Model(inputs=inputs,outputs=y)

> model.summary()
```

Layer (type)	Output Shape	Param #
=====		
input_2 (InputLayer)	(None, 16, 64)	0

lstm (LSTM)	(None, 32)	12416

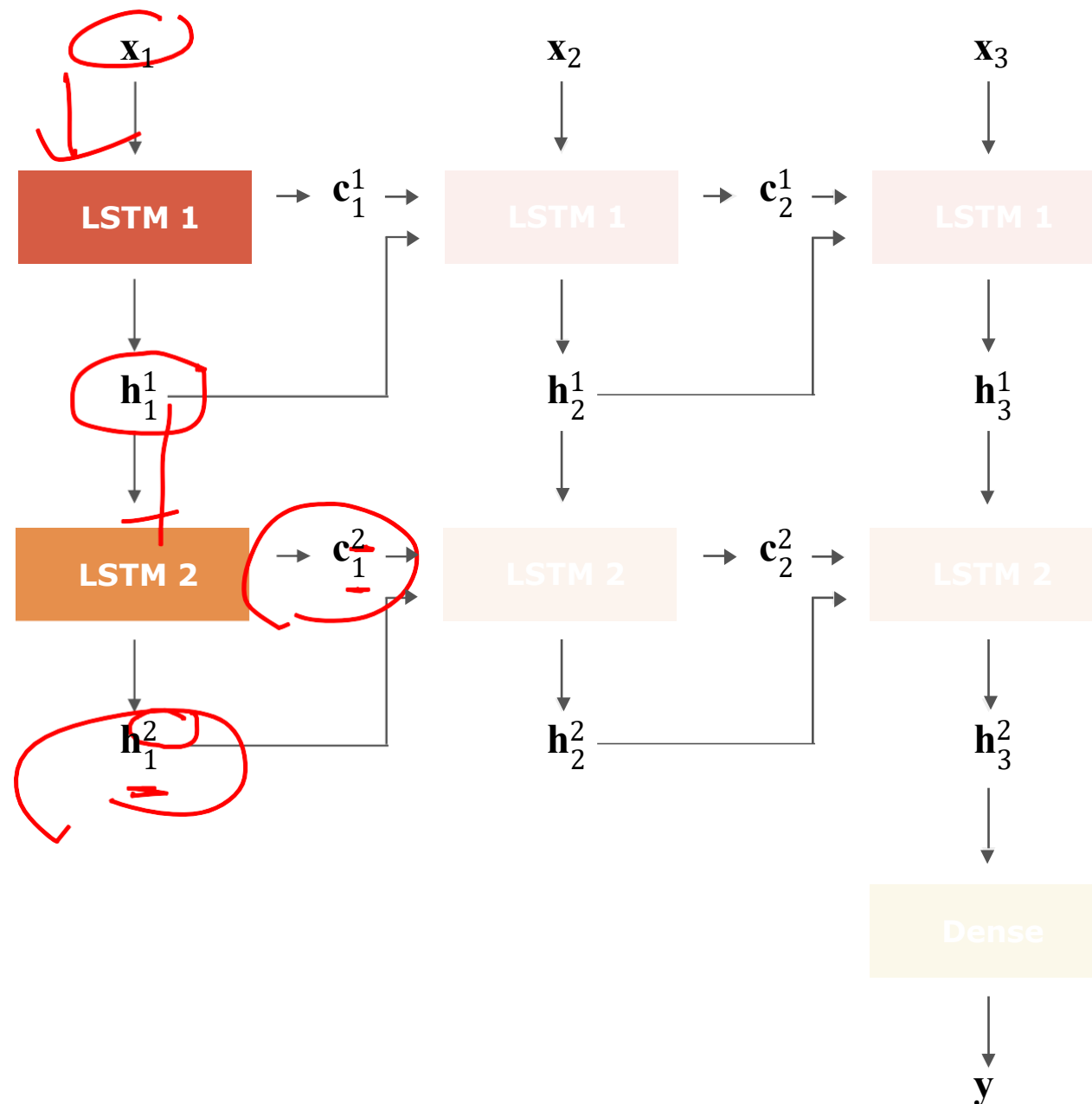
dense_1 (Dense)	(None, 1)	33
=====		
Total params: 12,449		
Trainable params: 12,449		
Non-trainable params: 0		

← Param # = 4×(U×(D+U))+U
where U is number of hidden neurons,
and D is number of features per time step

Stacking

Time step 1

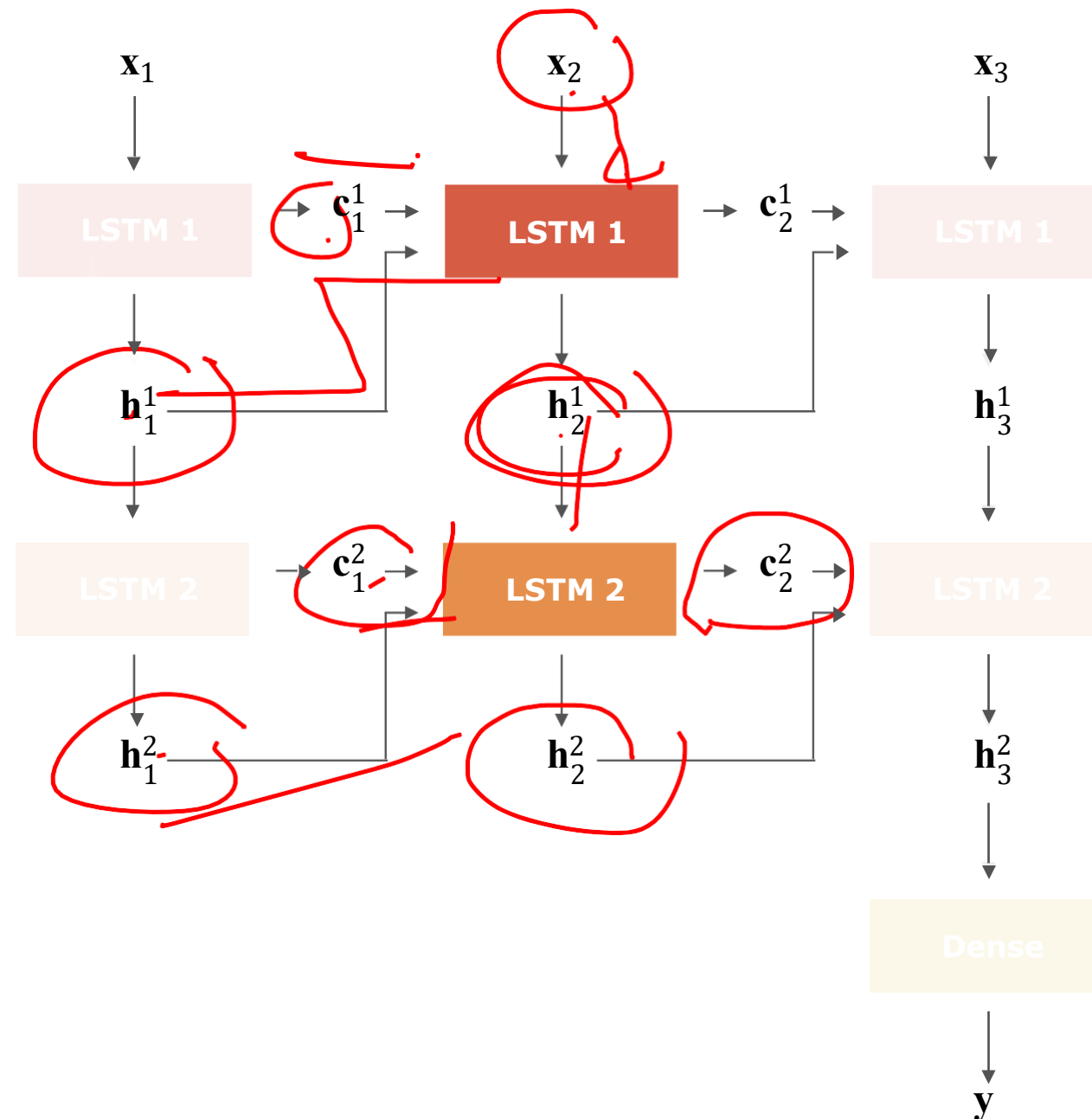
- It is possible to stack recurrent layers and make the network deeper
- Superscript denotes LSTM layer; Subscript denotes time step



Stacking

Time step 2

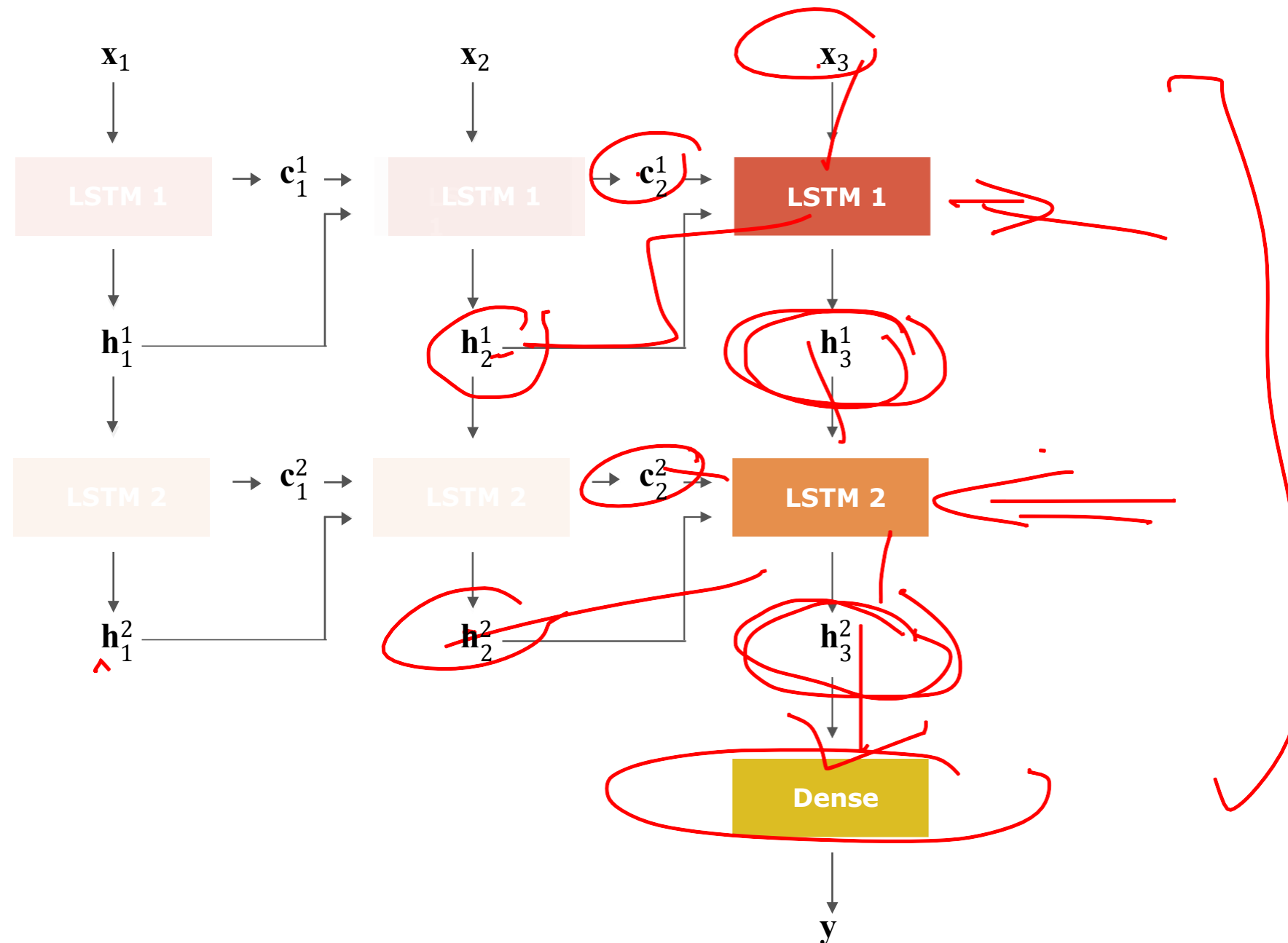
- Superscript denotes LSTM layer;
Subscript denotes time step



Stacking

Time step 3

- Superscript denotes LSTM layer;
Subscript denotes time step

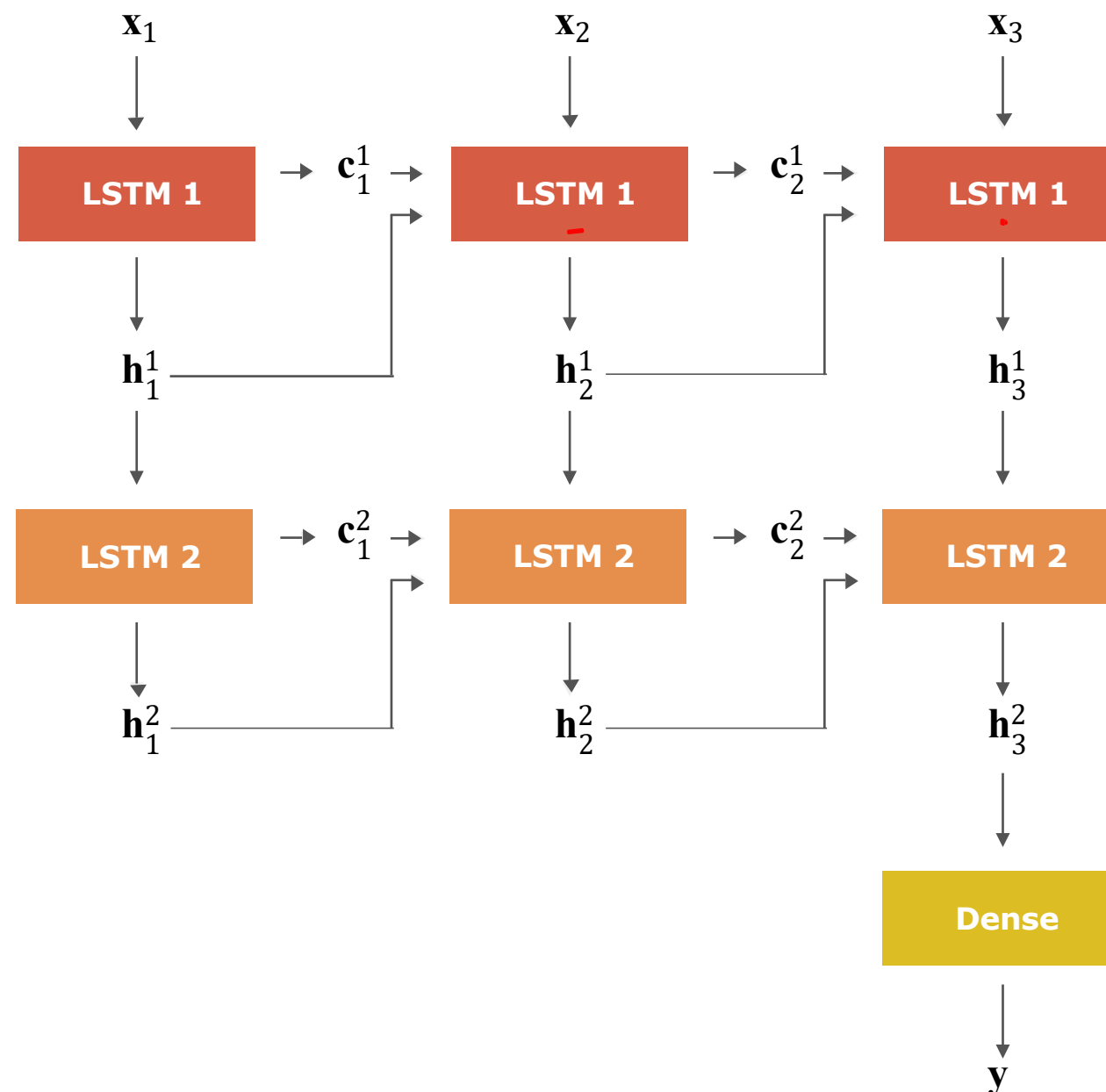


Stacking

On the output to Dense

- Superscript denotes LSTM layer;
Subscript denotes time step

- Note: for LSTM 2 to Dense, only the final output sequence is fed into the Dense layer



LSTM

How to build stacked LSTM in Keras

```
> inputs = Input(shape=(5,1))
> y = LSTM(7,return_sequences=True)(inputs)
> y = LSTM(9)(y)
> y = Dense(1, activation='linear')(y)

> model = Model(inputs=inputs,outputs=y)
> model.summary()
```

To stack LSTMs, return_sequences must be set to True.

The output layer determines the task. Use `Dense(1, activation='linear')` + MSE for **regression**, `Dense(1, activation='sigmoid')` + binary cross-entropy for **binary classification**, or `Dense(k, activation='softmax')` + categorical cross-entropy for **multi-class**.

Layer (type)	Output Shape	Param #
=====		
input_3 (InputLayer)	(None, 3, 5)	0

lstm_1 (LSTM)	(None, 3, 7)	364

lstm_2 (LSTM)	(None, 9)	612

dense_2 (Dense)	(None, 1)	10
=====		
Total params: 986		
Trainable params: 986		
Non-trainable params: 0		

→ The output shape is `(None, 3, 7)` because `return_sequences=True`. Otherwise, it would simply be `(None, 7)`. The number of timesteps in the output of an LSTM with `return_sequences=True` is always equal to the number of timesteps in the input.